Python Coding Standards

2.1 Code Formatting

- Follow **PEP 8** as the primary Python style guideline.
- Use consistent indentation of **4 spaces**.
- Do not use tabs for indentation.
- Keep lines reasonably short and readable.
- Use blank lines to separate logical sections of c

2.2 Imports

Imports should be organized into:

1. Standard library imports
2. Third-party libraries
3. Project-specific imports

Example:

```
import os
import subprocess

import psutil
from PySide6.QtWidgets import QApplication

from core.executor import Executor
from utils.logger import logger
```

Unused imports should be removed.

2.3 Functions

- Functions should be focused on a single responsibility.
- Function names should clearly describe their action.
- Functions should generally avoid excessive numbers of parameters.
- Return values should be predictable and documented when necessary.

Example:

```
def create_folder(path):
    ...
```

is preferred over:

```
def process(path, mode, flag, option):
    ...
```

when the function is specifically responsible for creating a folder.

2.4 Classes

- Classes should represent a clear concept or responsibility.
- Avoid creating classes simply to contain unrelated functions.
- Large classes should be divided when they begin handling multiple independent responsibilities.

Example:

```
class FileManager:
    ...
```

is preferable to a general-purpose class such as:

```
class SystemManager:
    # Files
    # Processes
    # Network
    # GUI
    # Database
```

2.6 Comments

Comments should explain **why** something is done when the reason is not obvious.

Avoid comments that simply repeat the code.

Bad:

```
# Increment count
count += 1
```

Better:

```
# Retry only temporary Windows process failures.
count += 1
```

2.7 Type Hints

Type hints should be used for important functions and public interfaces.

Example:

```
def create_folder(path: str) -> bool:
    ...
```

This is especially useful for communication between the core system and plugins.

2.8 Constants

Values that are reused or represent configuration should not be scattered throughout the code.

Example:

```
MAX_RETRIES = 3
DEFAULT_TIMEOUT = 10
```

rather than repeatedly using unexplained numbers.

Naming Conventions

Consistent naming is required throughout the project.

3.1 Variables

Use `snake_case`.

```
file_path
process_name
command_result
```

Avoid:

```
filePath
ProcessName
f
```

unless a very short local variable is appropriate.

3.2 Functions

Use `snake_case` and start with a descriptive verb where appropriate.

```
create_folder()
open_application()
get_system_info()
execute_command()
```

3.3 Classes

Use `PascalCase`.

```
FileManager
CommandParser
ExecutionEngine
PluginManager
SystemMonitor
```